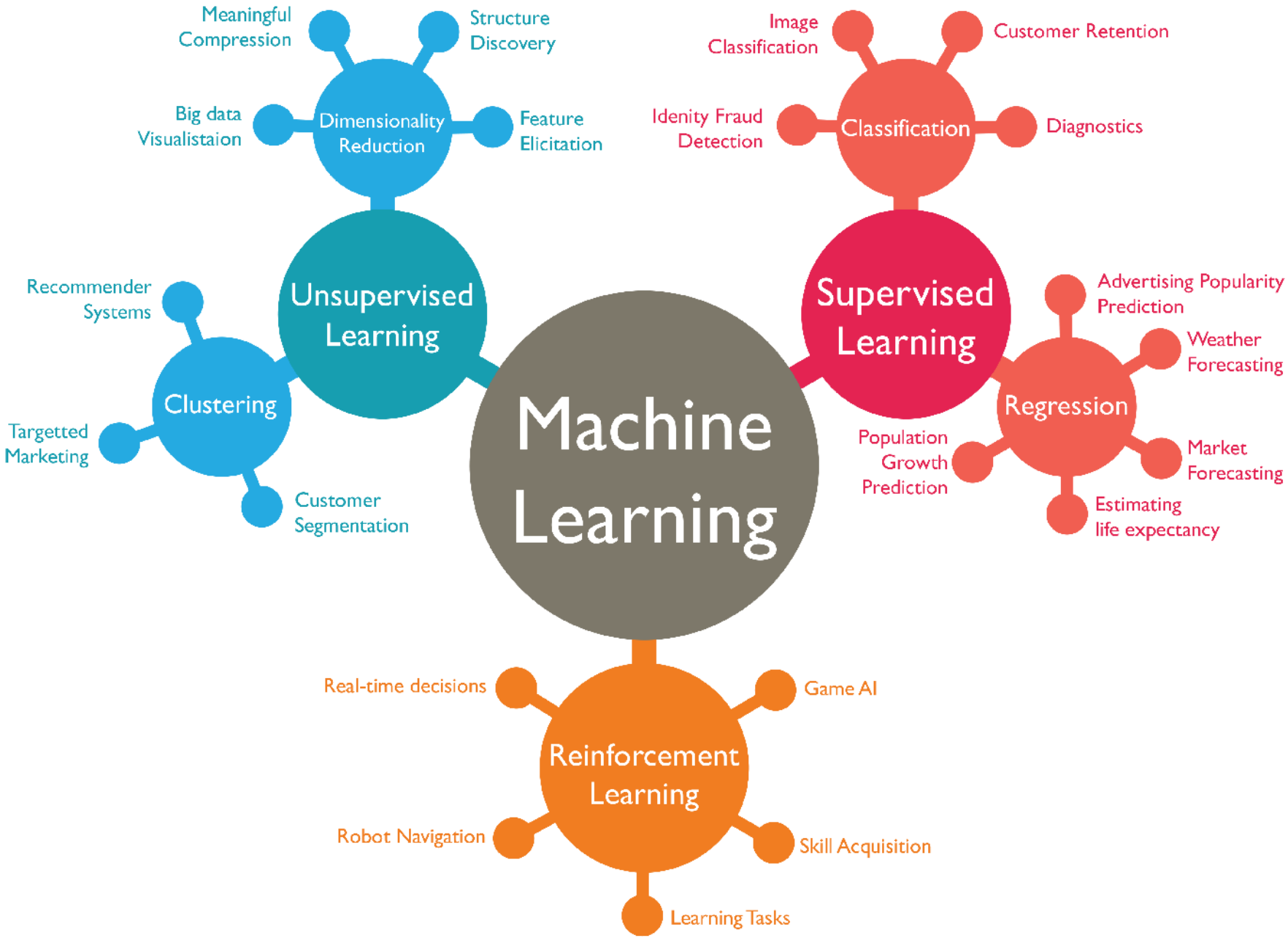
SENG 499: Fundamentals of Data Sciences

Chapter 5: Machine Learning

This Chapter is prepared by Dr. Abdullah Almessmar

Machine Learning

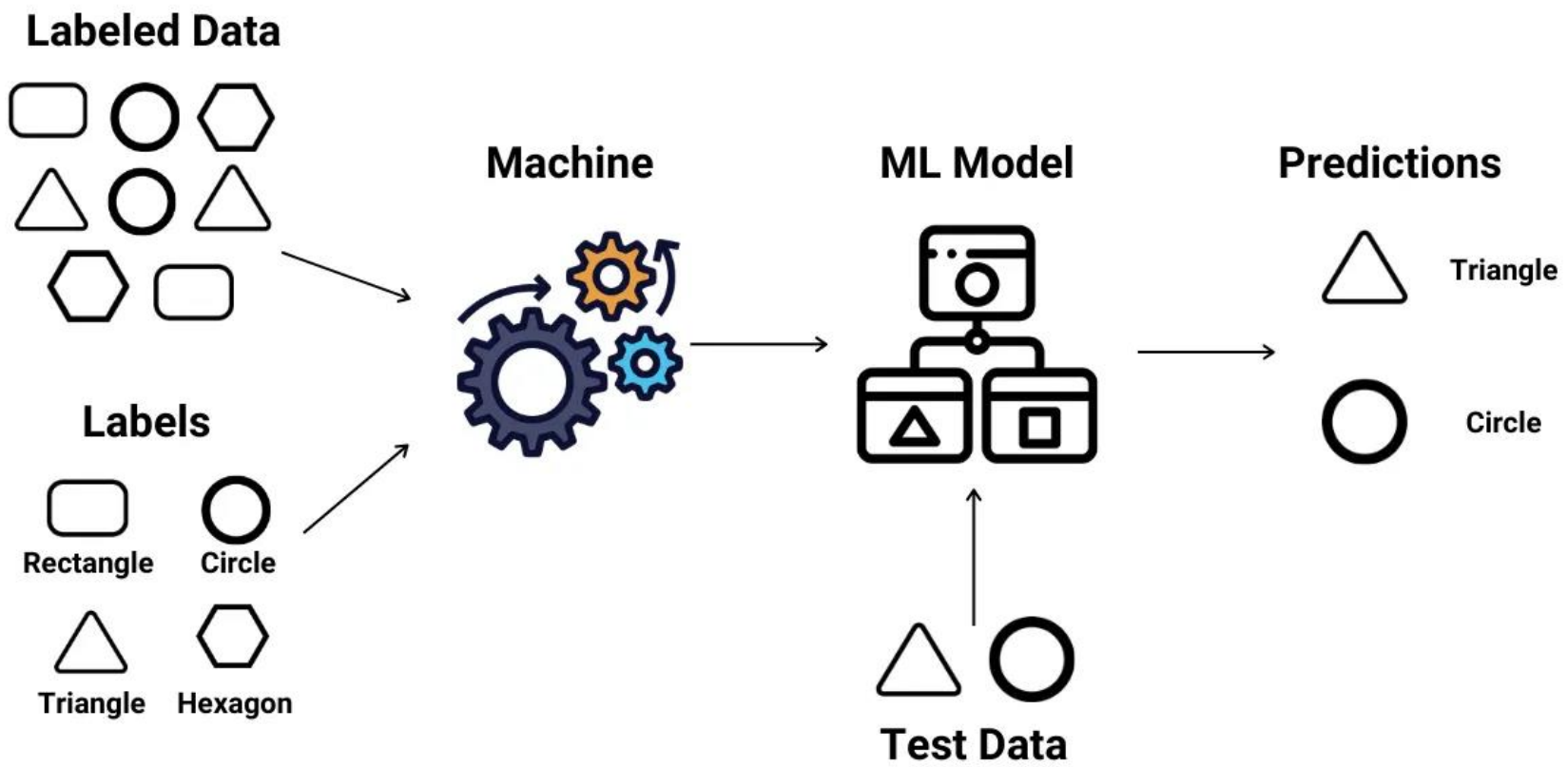
- **Machine Learning** involves coding programs that automatically adjust their performance in accordance with their exposure to information in data.
- This learning is achieved via a parameterized model with tunable parameters that are automatically adjusted according to different performance criteria.
- Machine learning can be considered a subfield of artificial intelligence (AI)


Machine Learning
Unsupervised Learning
Dimensionality Reduction
Meaningful Compression
Structure Discovery
Big data Visualistaion
Feature Elicitation
Clustering
Recommender Systems
Targetted Marketing
Customer Segmentation
Supervised Learning
Classification
Image Classification
Customer Retention
Idenity Fraud Detection
Diagnostics
Regression
Advertising Popularity Prediction
Weather Forecasting
Population Growth Prediction
Market Forecasting
Estimating life expectancy
Reinforcement Learning
Real-time decisions
Game AI
Robot Navigation
Skill Acquisition
Learning Tasks

1. Supervised Learning

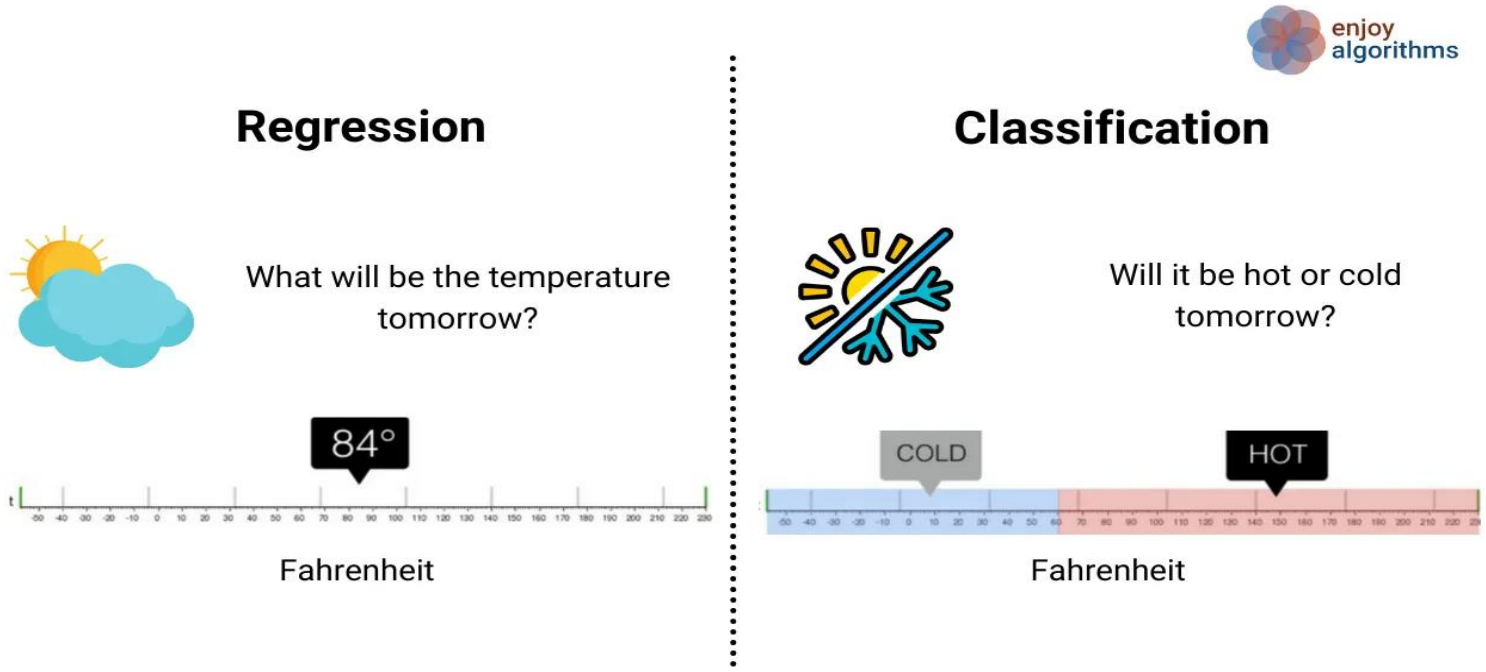
- In Supervised Learning, the algorithm is trained on a labeled dataset, which means the input data is paired with corresponding output labels.
- The goal is for the algorithm to learn a mapping from the input to the output, making predictions or classifications based on the provided labeled examples.
- The main tasks of this type is: Classification and Regression

Supervised Learning



Supervised Learning Types

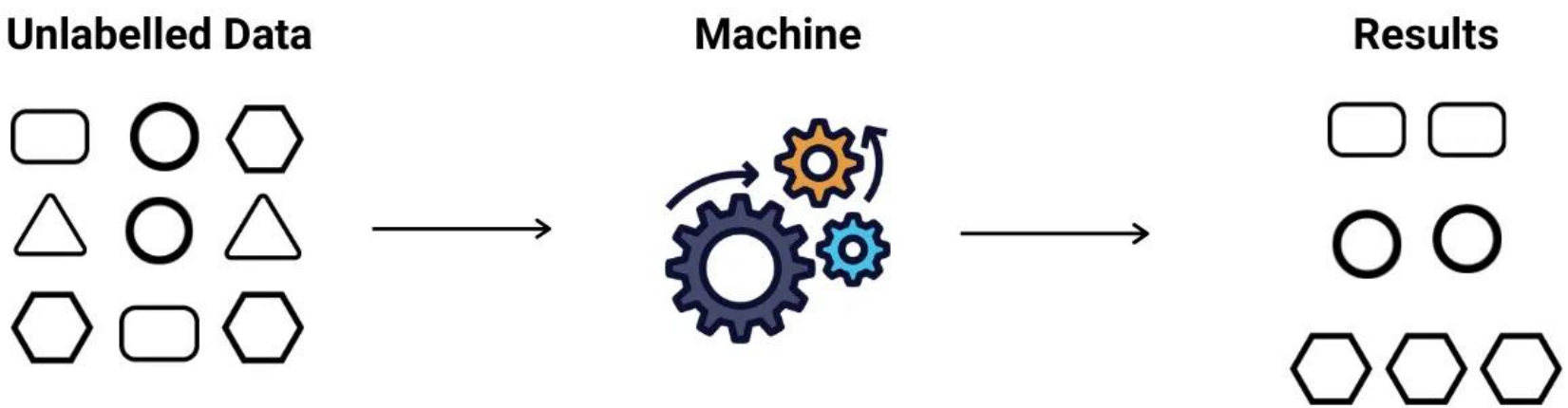
- 1. **Classification:** where our result set consist of categories
- 2. **Regression:** where the results are continuous values.



2. Unsupervised Learning

- Unsupervised Learning involves training an algorithm on an unlabeled dataset, where the algorithm must find patterns and relationships within the data on its own.
- In contrast to supervised learning, no output labels are pre-determined. Instead, the algorithm explores the inherent structure of the data, often through clustering or dimensionality reduction.
- Common tasks include clustering similar data points or reducing the complexity of the data.

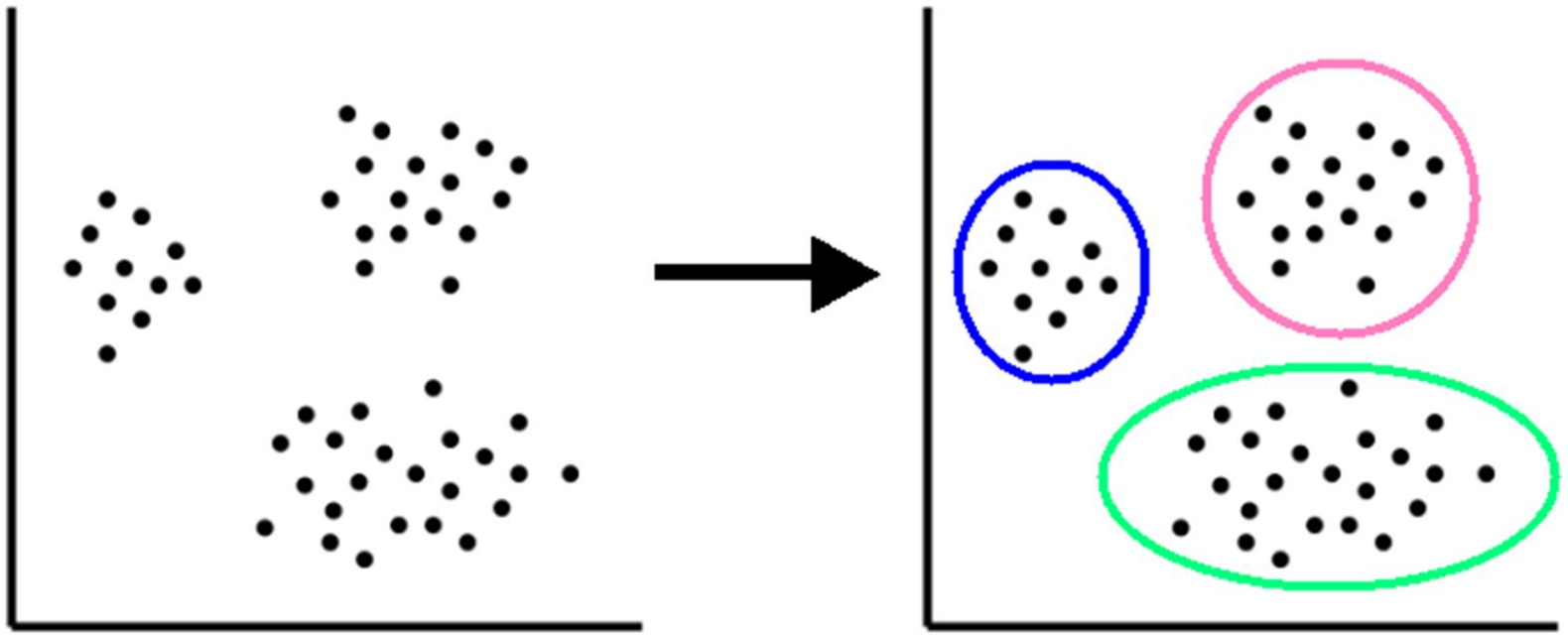
Unsupervised Learning



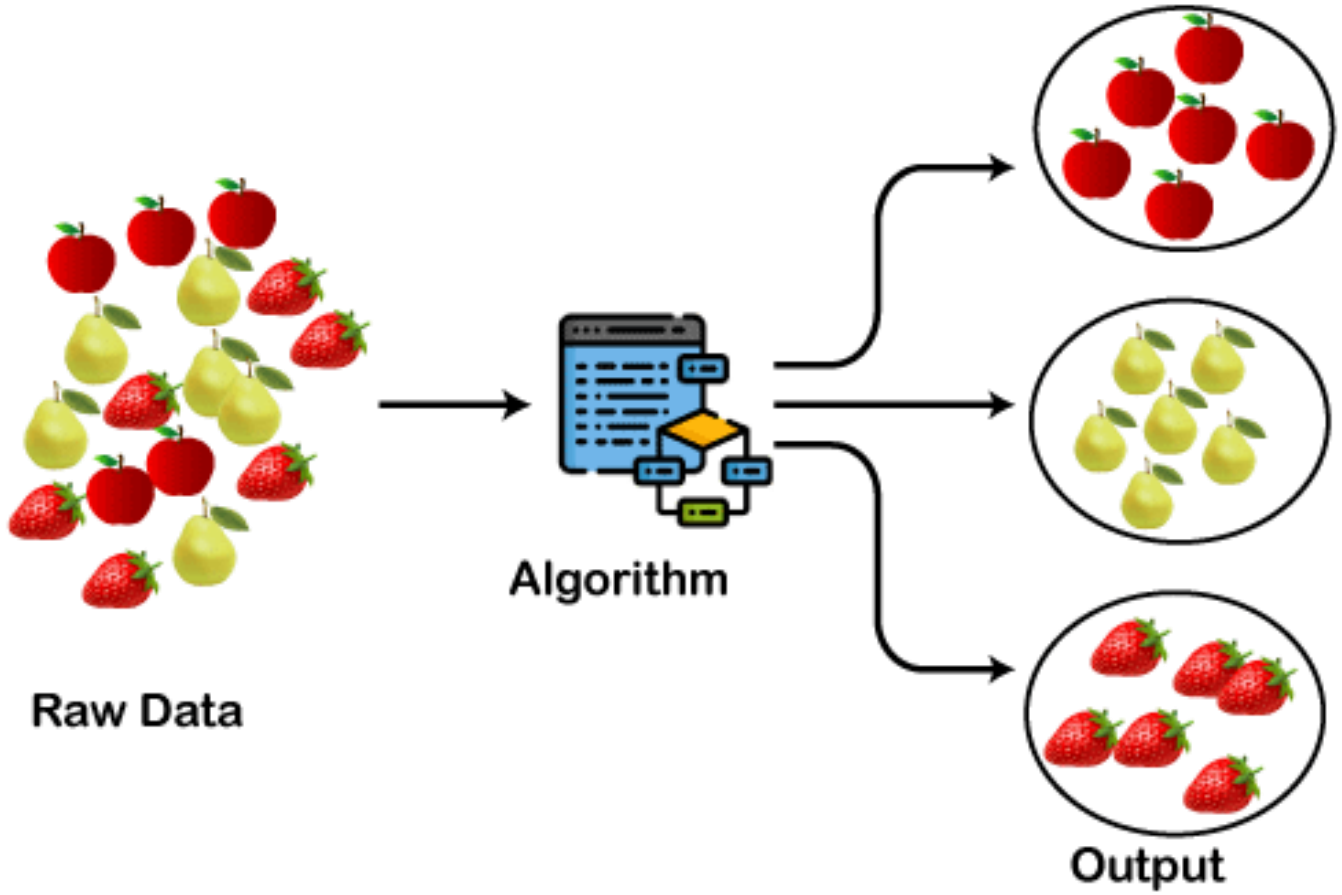
Unsupervised Learning Types

- 1. **Clustering:** where data is grouped in a meaningful way.
- 2. **Dimensionality Reduction:** where high-dimensional data is represented with low-dimensional data
- 3. **Association:** where the relationships between variables in a big dataset is discovered.

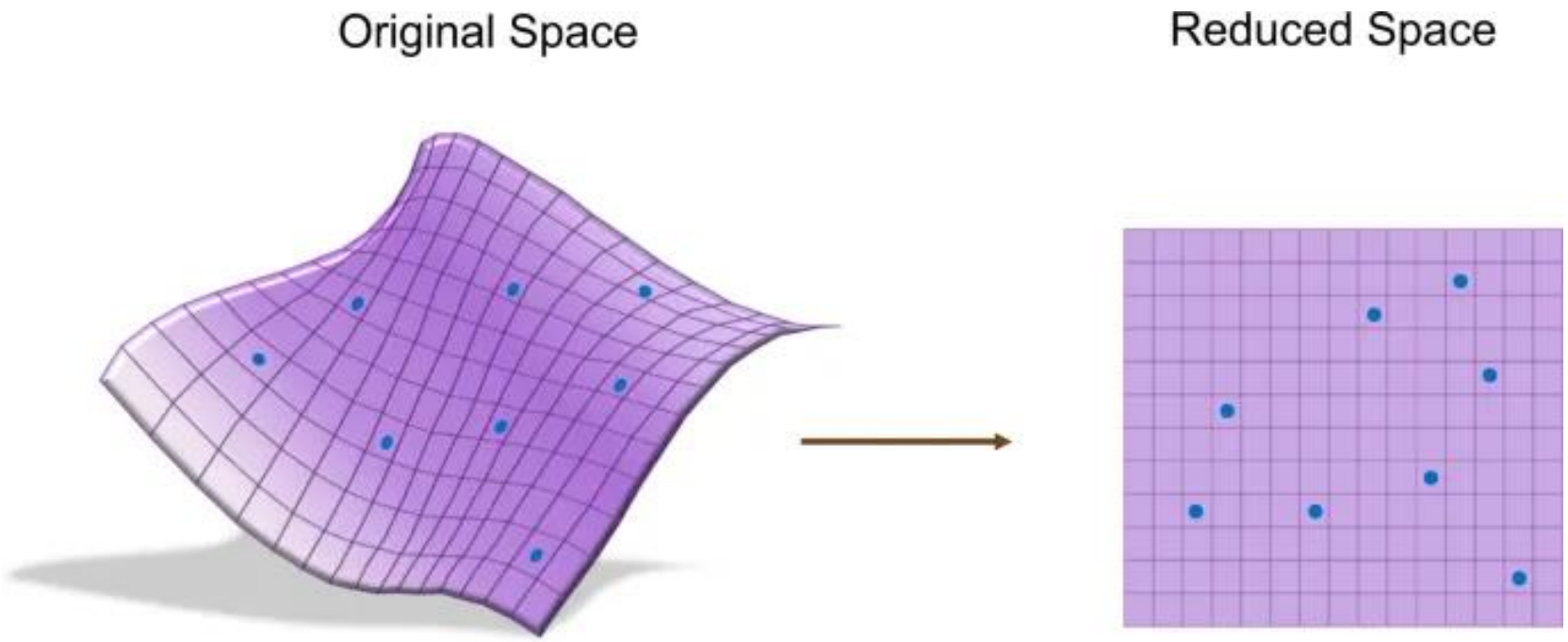
Clustering



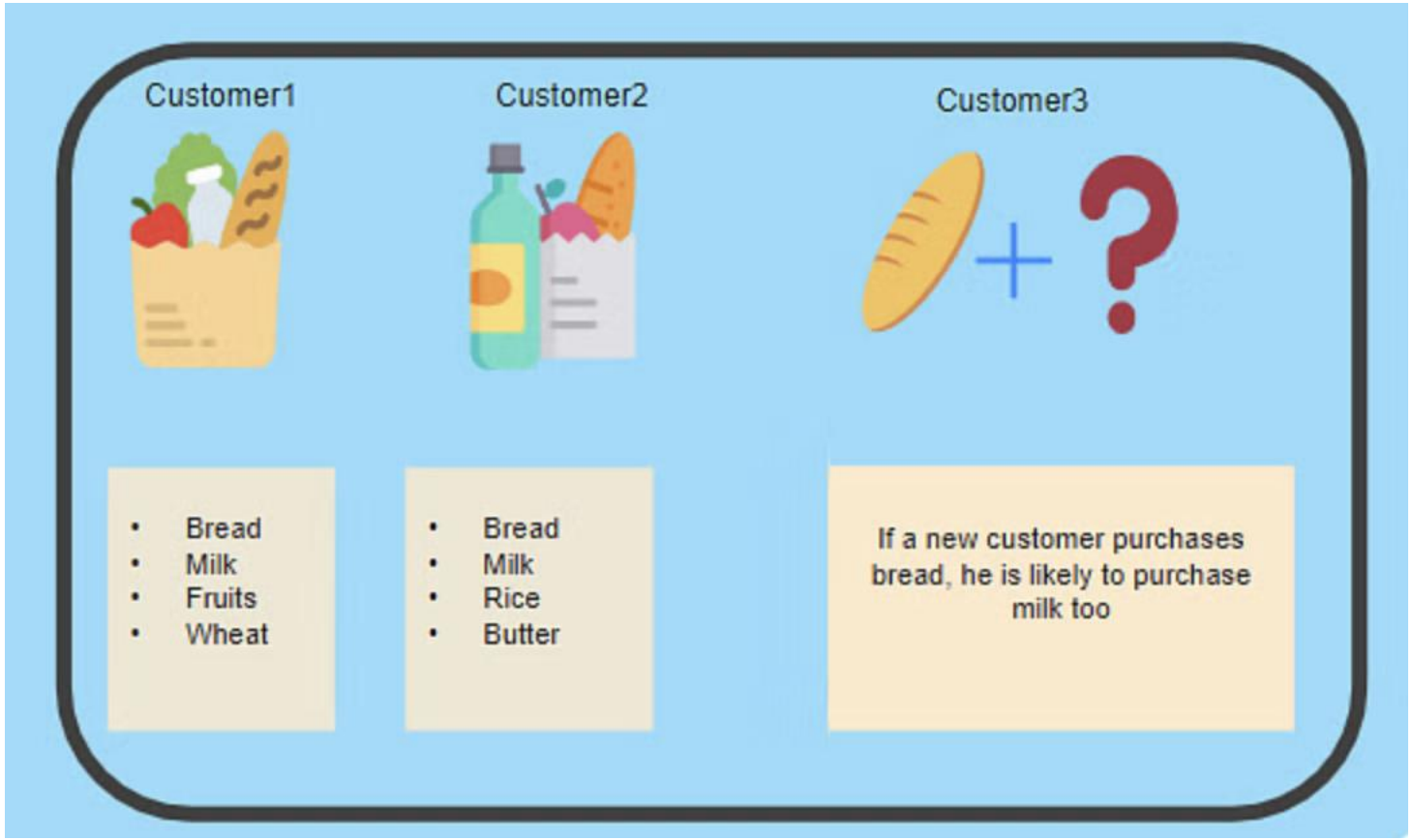
Clustering



Dimentional reduction

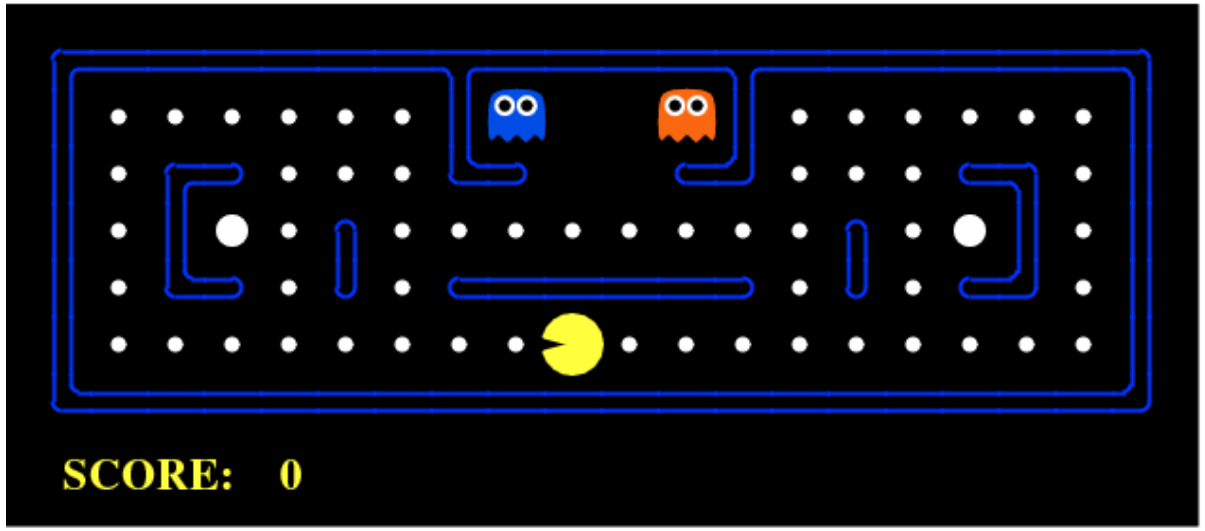


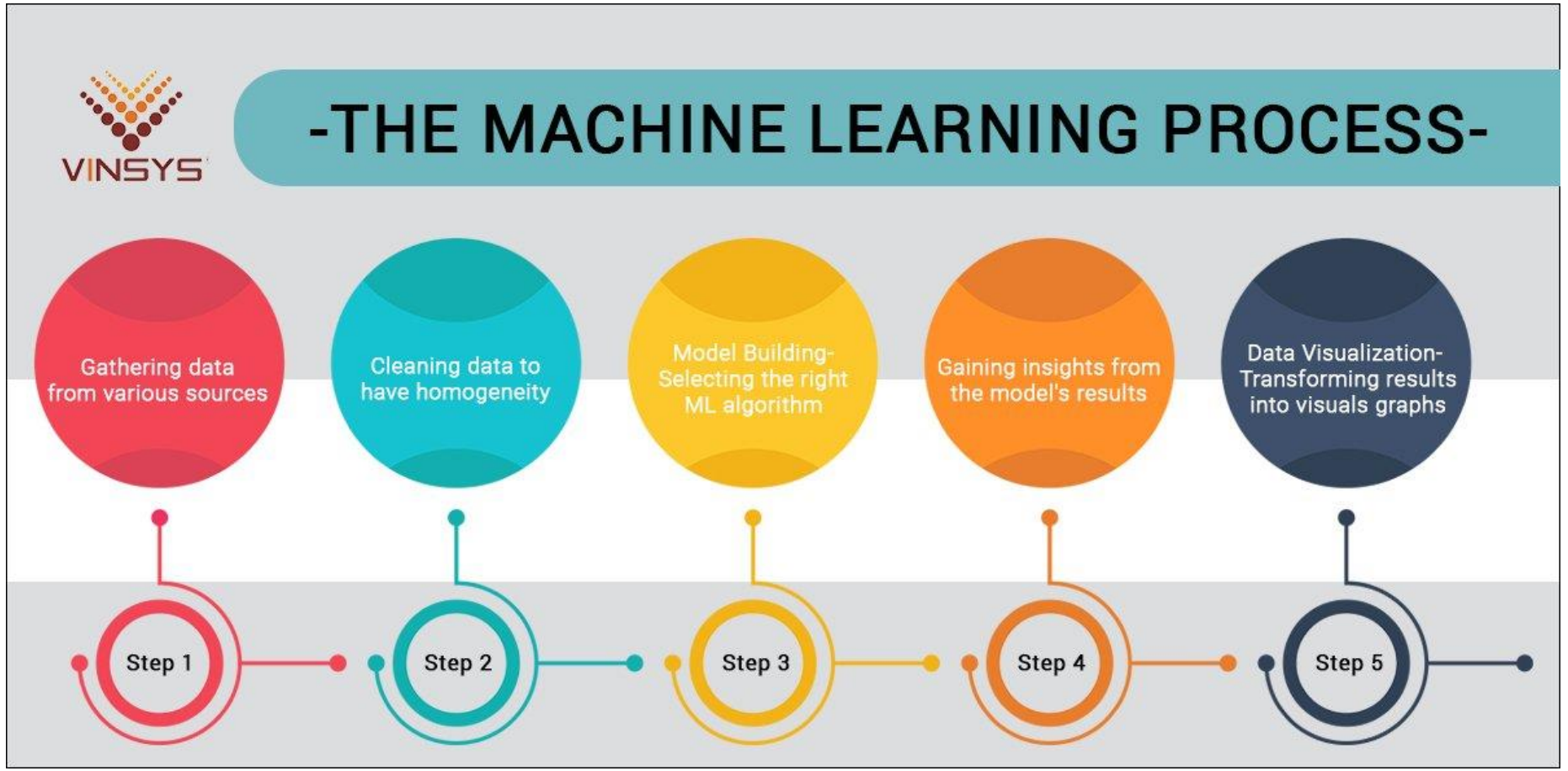
Association



3. Reinforcement Learning

- Algorithms that learn via reinforcement from criticism that provides information on the quality of a solution, but not on how to improve it.
- Improved solutions are achieved by iteratively exploring the solution space.




VINSYS
-THE MACHINE LEARNING PROCESS-
Gathering data from various sources
Cleaning data to have homogeneity
Model Building- Selecting the right ML algorithm
Gaining insights from the model's results
Data Visualization- Transforming results into visuals graphs
Step 1
Step 2
Step 3
Step 4
Step 5

Classification

Classification

- As a data scientist, the first step you apply given a certain problem is to identify the question to be answered. According to the type of answer we are seeking, we are directly aiming for a certain set of techniques.
- If our question is answered by **YES/NO**, we are facing a **classification** problem.

Classification

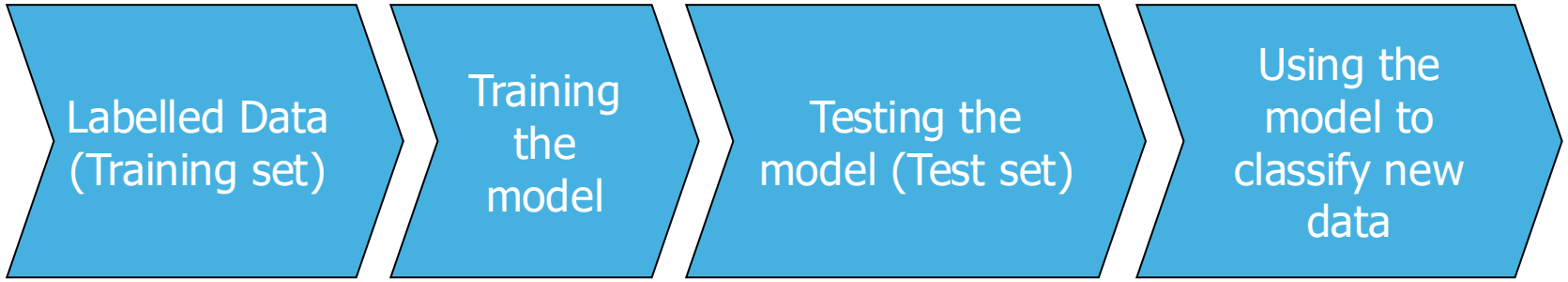
Classifiers are also the tools to use if our question admits only a discrete set of answers, i.e., we want to select from a finite number of choices.

- Examples:
  - Given the results of a clinical test, e.g., does this patient suffer from diabetes?
  - Given a magnetic resonance image, is it a tumor shown in the image?
  - Given the past activity associated with a credit card, is the current operation fraudulent?

Classification

- If our question is a **prediction of a real-valued quantity**, we are faced with a **regression problem**. We will go into details of regression in the next chapter.
- Examples:
  - - Given the description of an apartment, what is the expected market value of the flat? What will the value be if the apartment has an elevator?
  - - Given the past records of user activity on Apps, how long will a certain client be connected to our App?
  - - Given my skills and marks in computer science and math, what mark will I achieve in a data science course?

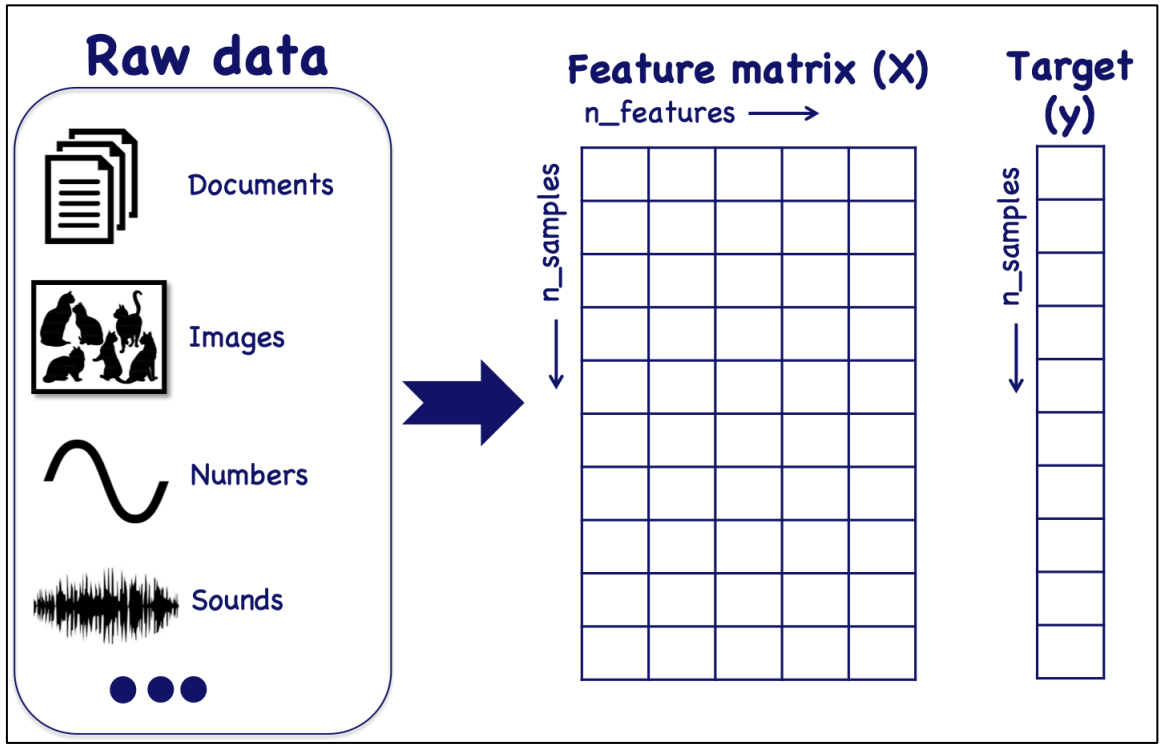
Basic Steps for Classification



Scikit-learn

- A problem in Scikit-learn is modeled as follows:
- Input data is structured in Numpy arrays. The size of the array is expected to be **[n_samples, n_features]:**
  - **n_samples**: The number of samples (n). Each sample is an item to process (e.g., classify). A sample can be a document, a picture, an audio file, a video, an astronomical object, a row in a database or CSV file, or whatever you can describe with a fixed set of quantitative traits.
  - **n_features**: The number of features (d) or distinct traits that can be used to describe each item in a quantitative manner. Features are generally real-valued, but may be Boolean, discrete-valued or even categorical.

General Concept



Numpy Structure

$$\text{feature matrix}: \quad \mathbf{X} = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1d} \\ x_{21} & x_{22} & \cdots & x_{2d} \\ x_{31} & x_{32} & \cdots & x_{3d} \\ \vdots & \vdots & \ddots & \vdots \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{nd} \end{bmatrix}$$

$$\text{label vector}: \quad \mathbf{y}^T = [y_1, y_2, y_3, \cdots y_n]$$

Performance metrics

- **Accuracy:** The basic measure of performance of a classifier. This is defined as the number of correctly predicted examples divided by the total amount of examples.
- Accuracy is related to the error as follows: acc = 1 − err.

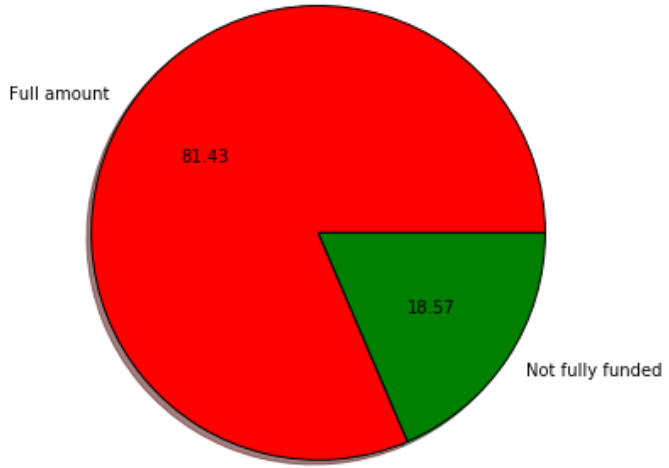
$$acc = \frac{\textbf{Number of correct predictions}}{n}$$

- Each estimator has a score() method that invokes the default scoring metric.
- Example: In the case of k-nearest neighbors, this is the classification accuracy.

```
knn.score(x,y)
```

Performance metrics

- If the accuracy was = 0.8316 , It looks like a really good result. But how good is it?
- If we check the distribution of the labels as in figure:

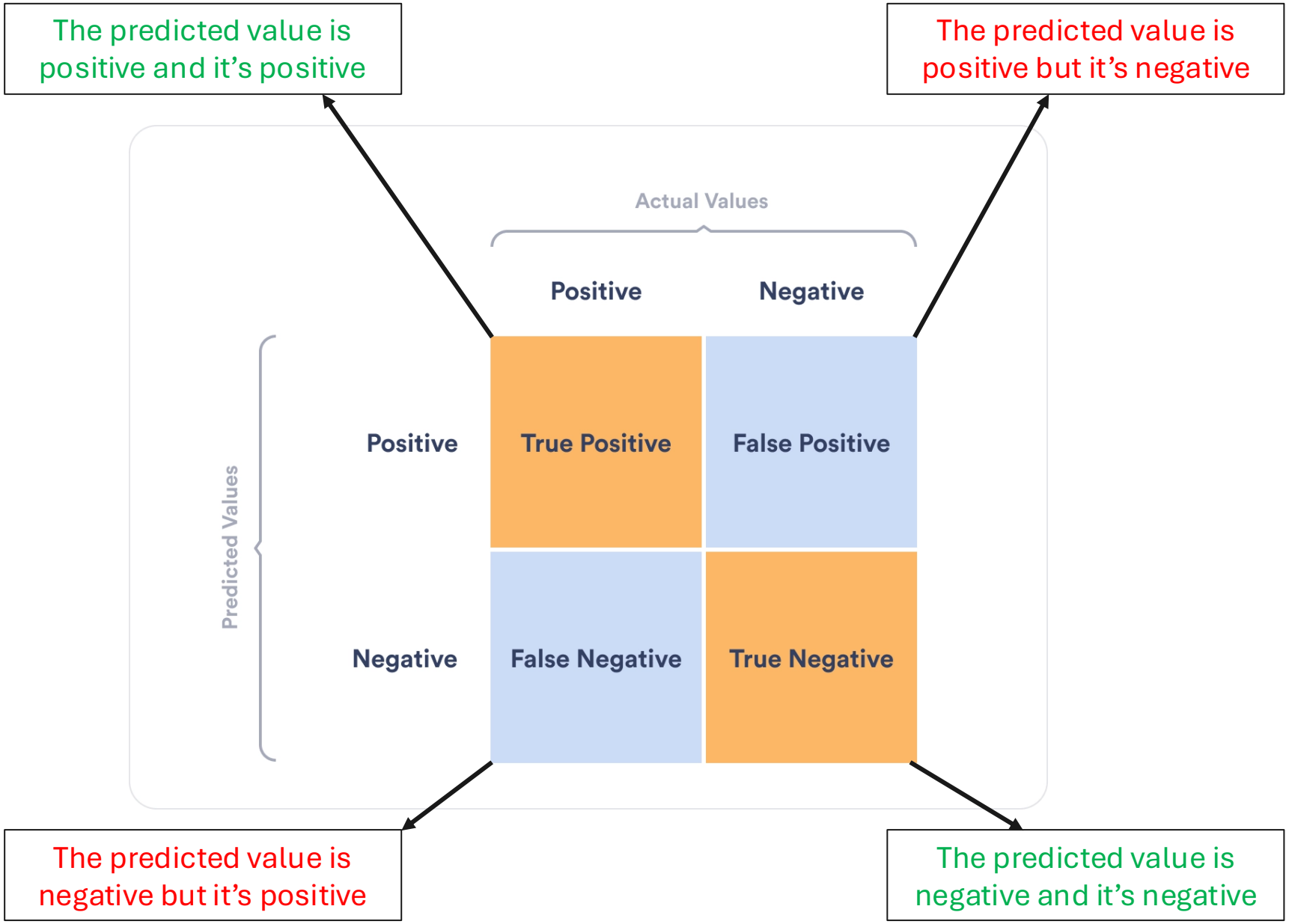


Performance metrics

- Note that there are far more positive labels than negative ones.
- In this case, the dataset is referred to as unbalanced.
- a very simple rule such as always predict the majority class, will give us good performance.
- In our problem, always predicting that the loan will be fully funded correctly predicts 81.57% of the samples. Observe that this value is very close to that obtained using the classifier.
- **Is still the accuracy a good performance metric?**

Confusion matrix

- The confusion matrix enables us to define different metrics considering such scenarios.
- The confusion matrix considers the concepts of the classifier outcome and the actual ground truth or gold standard.
- In a binary problem, there are four possible cases:
  - **True positives (TP):** When the classifier predicts a sample as positive and it really is positive.
  - **False positives (FP):** When the classifier predicts a sample as positive but in fact it is negative.
  - **True negatives (TN):** When the classifier predicts a sample as negative and it really is negative.
  - **False negatives (FN):** When the classifier predicts a sample as negative but in fact it is positive.


The predicted value is positive and it's positive
The predicted value is positive but it's negative
Actual Values
Positive
Negative
Predicted Values
Positive
True Positive
False Positive
Negative
False Negative
True Negative
The predicted value is negative but it's positive
The predicted value is negative and it's negative

Confusion Matrix

- The combination of these elements allows us to define several performance metrics:
- Accuracy:

$$accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

- ‘Column-wise’ we find these two partial performance metrics:
  - Sensitivity or Recall:

$$sensitivity = \frac{TP}{Real\ Positives} = \frac{TP}{TP + FN}$$

  - Specificity:

$$specificity = \frac{TN}{Real\ Negative} = \frac{TN}{TN + FP}$$

Confusion Matrix

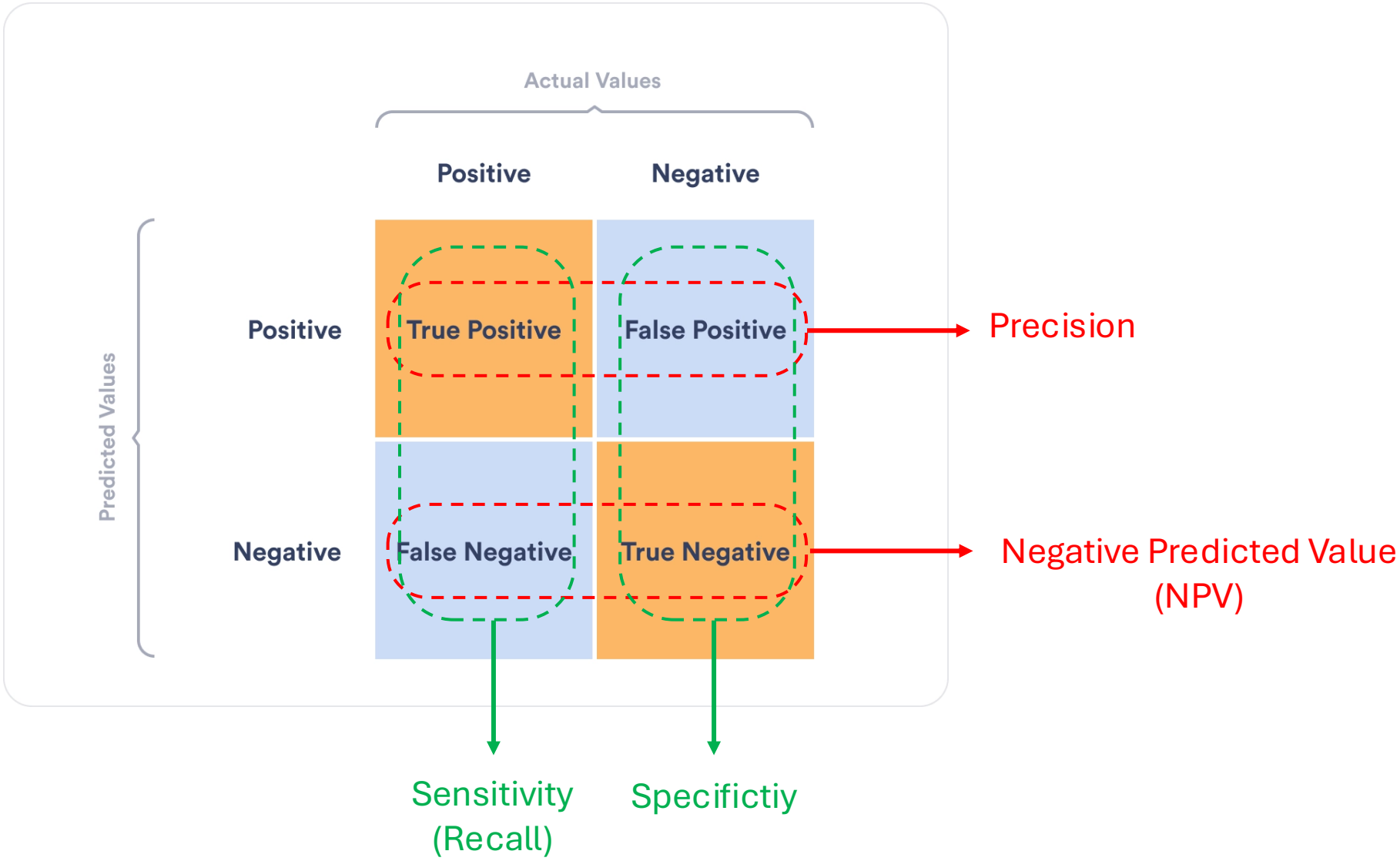
- ‘Row-wise’ we find these two partial performance metrics:
  - Precision or Positive Predictive Value:

$$\text{precision} = \frac{TP}{\text{Predicted Positives}} = \frac{TP}{TP + FP}$$

  - Negative Predictive Value (NPV)

$$\text{NPV} = \frac{TN}{\text{Predicted Negative}} = \frac{TN}{TN + FN}$$

- what is the rate of predictions for not fully funded loans that have actually not been fully funded? **This question is answered by recall**
- Of all the fully funded loans predicted by the classifier, how many have been fully funded? **This is answered by the precision metric.**


Actual Values
Positive
Negative
Predicted Values
Positive
Negative
True Positive
False Positive
False Negative
True Negative
Precision
Negative Predicted Value
(NPV)
Sensitivity
(Recall)
Specifictiy

Example: Spam Detection

- **Scenario:** An email service provider uses a spam detection algorithm to filter out unwanted emails from users' inboxes. The algorithm analyzes incoming emails and classifies them as either "Spam" or "Not Spam" based on various indicators such as sender reputation, keywords, and patterns in the content.

- **True Positive (TP):** The algorithm correctly identifies a spam email. This means the email is flagged as spam and moved to the spam folder, preventing the user from seeing unwanted emails in their inbox.

Example: Spam Detection

- **True Negative (TN):** The algorithm correctly identifies a legitimate email as not spam. The email lands in the user's inbox as intended, ensuring important communications are not missed.

- **False Positive (FP):** The algorithm incorrectly flags a legitimate email as spam. This is a false alarm, causing potentially important emails to be moved to the spam folder, where the user might overlook them.

- **False Negative (FN):** The algorithm fails to identify a spam email, allowing it to appear in the user's inbox. This outcome means the user is exposed to potentially harmful or unwanted content.

Example: Spam Detection

- **Accuracy** measures how well the spam detection algorithm correctly identifies both spam and not spam emails.
- **Precision** assesses the proportion of emails flagged as spam that are actually spam. High precision means that when an email is flagged as spam, there's a high chance it truly is spam, minimizing the risk of losing important emails.
- **Recall (Sensitivity)** evaluates how good the algorithm is at catching all the spam emails. High recall is essential to ensure that most spam emails are correctly identified and filtered out, keeping the user's inbox clean.
- **Specificity** measures the algorithm's ability to correctly identify not spam emails. High specificity means that not spam emails are rarely misclassified as spam, ensuring important messages reach the inbox.

- Imagine over a certain period, our email spam detection algorithm processes 1000 emails. Here are the actual values of dataset:
  - 600 emails are actual spam.
  - 400 emails are legitimate (not spam).
- But The output of the algorithm's classifications are as follows:
  - 500 emails are correctly identified as spam (True Positives, TP).
  - 300 emails are correctly identified as legitimate (True Negatives, TN).
  - 80 emails are legitimate emails incorrectly marked as spam (False Positives, FP).
  - 120 emails are spam emails that are missed and land in the inbox (False Negatives, FN).

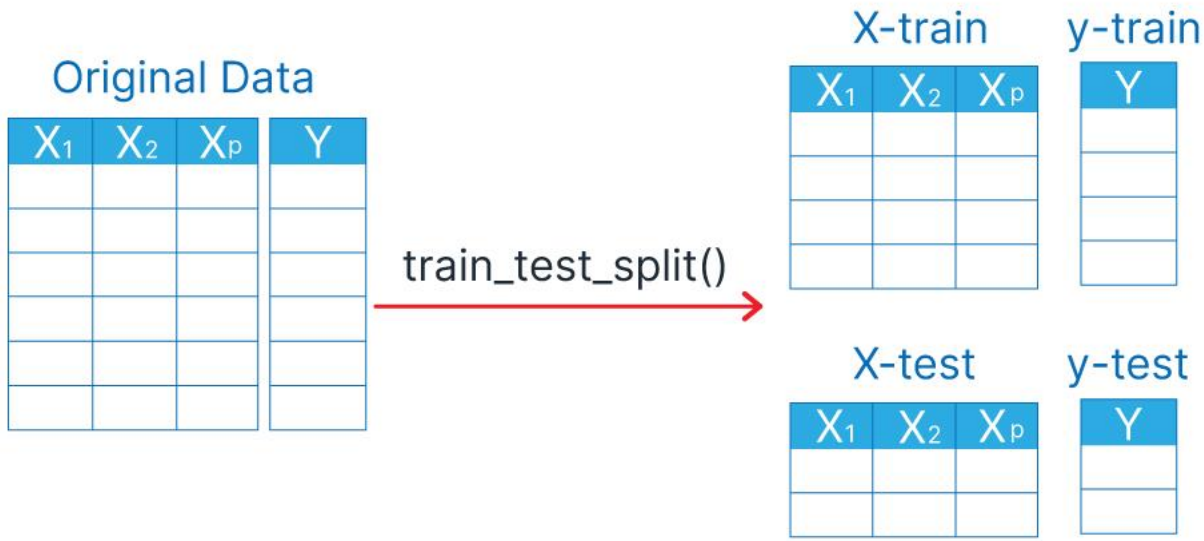
- $accuracy = \frac{500+300}{500+300+80+120}$
- $sensitivity = \frac{500}{500+120}$
- $specificity = \frac{300}{300+80}$
- $\text{precision} = \frac{500}{500+80}$
- $\text{NPV} = \frac{300}{300+120}$

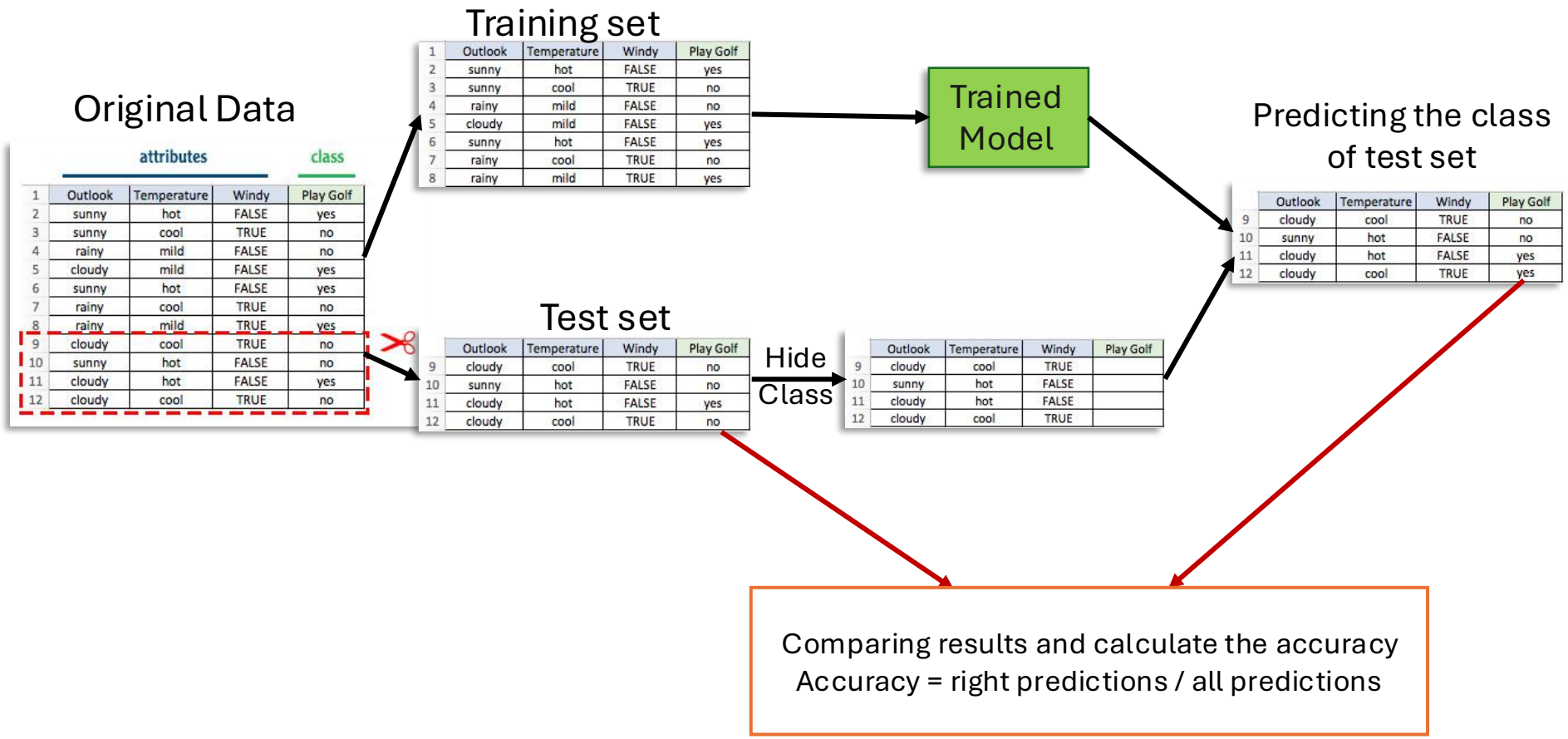
General Model selection techniques

- Splitting the dataset into training and testing sets or using cross-validation techniques is a fundamental practice in machine learning to evaluate the performance of a model.
- These techniques aim to ensure that the model can generalize well to unseen data, avoiding problems like overfitting or underfitting.

1. Training and Testing Split

- The dataset is divided into two parts: one for training the model and the other for testing.
- The training set is used to fit the model, while the testing set is used to evaluate its generalization capability.
- The split can be 70/30, 80/20, or any other ratio depending on the dataset size and the specific problem.
- This method is simple and easy to implement but has some limitations, primarily the risk of high variance if the split doesn't represent the overall dataset well.


Original Data
X1
X2
Xp
Y
train_test_split()
X-train
y-train
X-test
y-test



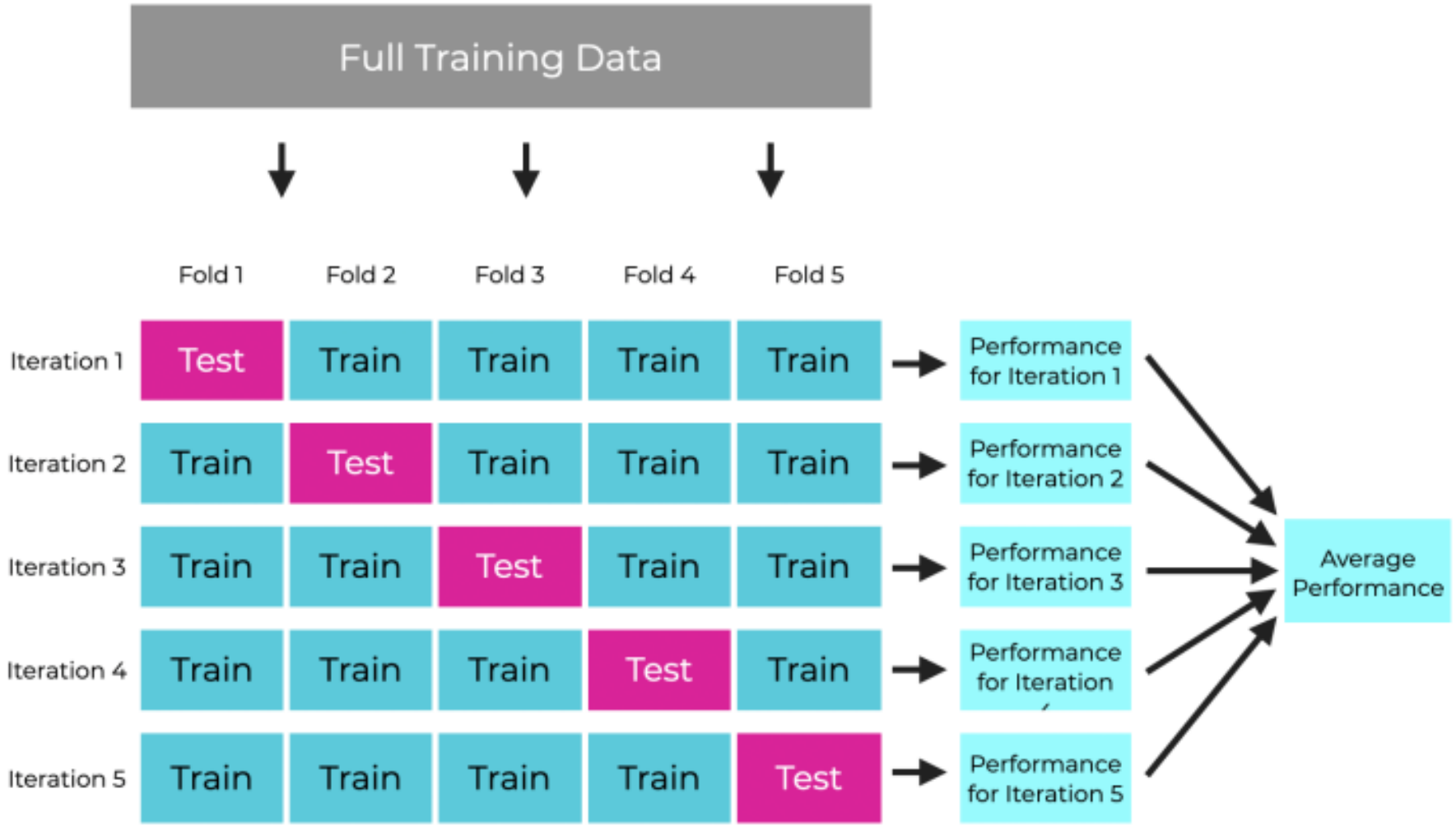
	Outlook	Temperature	Windy	Play Golf
1	sunny	hot	FALSE	yes
2	sunny	cool	TRUE	no
3	rainy	mild	FALSE	no
4	cloudy	mild	FALSE	yes
5	sunny	hot	FALSE	yes
6	rainy	cool	TRUE	no
7	rainy	mild	TRUE	yes
8	cloudy	cool	TRUE	no
9	sunny	hot	FALSE	no
10	cloudy	hot	FALSE	yes
11	cloudy	cool	TRUE	no

2. Cross Validation

- To overcome the limitations of a simple train-test split, cross-validation is often used. The most common method is k-fold cross-validation:
  - K-Fold Cross-Validation.
  - Leave-One-Out Cross-Validation.

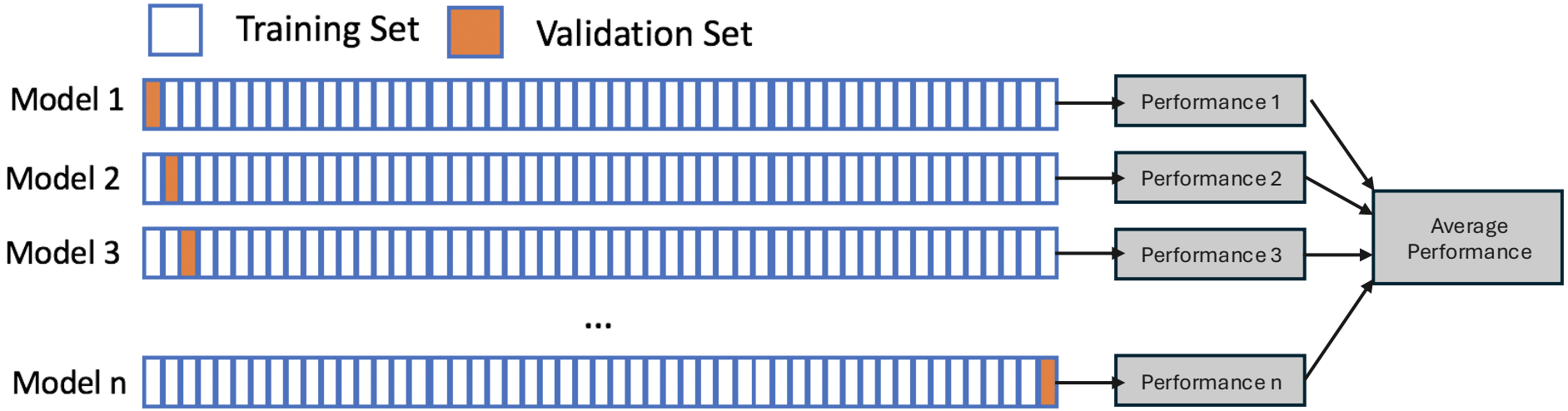
K-Fold Cross-Validation

- The dataset is divided into k equal-sized folds.
- The model is trained k times, each time using a different fold as the testing set and the remaining k-1 folds combined as the training set.
- The performance measure reported is the **average** of the values computed in the loop.
- This method is beneficial because it ensures that every observation from the original dataset has the chance of appearing in the training and test set. It is especially useful when the dataset is not large enough to provide a good representation of the data across multiple splits.


Full Training Data
Fold 1
Fold 2
Fold 3
Fold 4
Fold 5
Iteration 1
Test
Train
Train
Train
Train
Performance for Iteration 1
Iteration 2
Train
Test
Train
Train
Train
Performance for Iteration 2
Iteration 3
Train
Train
Test
Train
Train
Performance for Iteration 3
Iteration 4
Train
Train
Train
Test
Train
Performance for Iteration
Iteration 5
Train
Train
Train
Train
Test
Performance for Iteration 5
Average Performance

Leave-One-Out Cross-Validation

- A special case of k-fold cross-validation where **k** equals the number of data samples (points) in the dataset. Essentially, the model is trained **n** times (**n** being the number of data samples) using **n-1** data samples for training and the remaining one for testing.
- This method can be very computationally expensive for large datasets but is useful for small datasets as it maximizes the training data usage.


Training Set
Validation Set
Model 1
Model 2
Model 3
...
Model n
Performance 1
Performance 2
Performance 3
Performance n
Average Performance

How to Choose the Right Method?

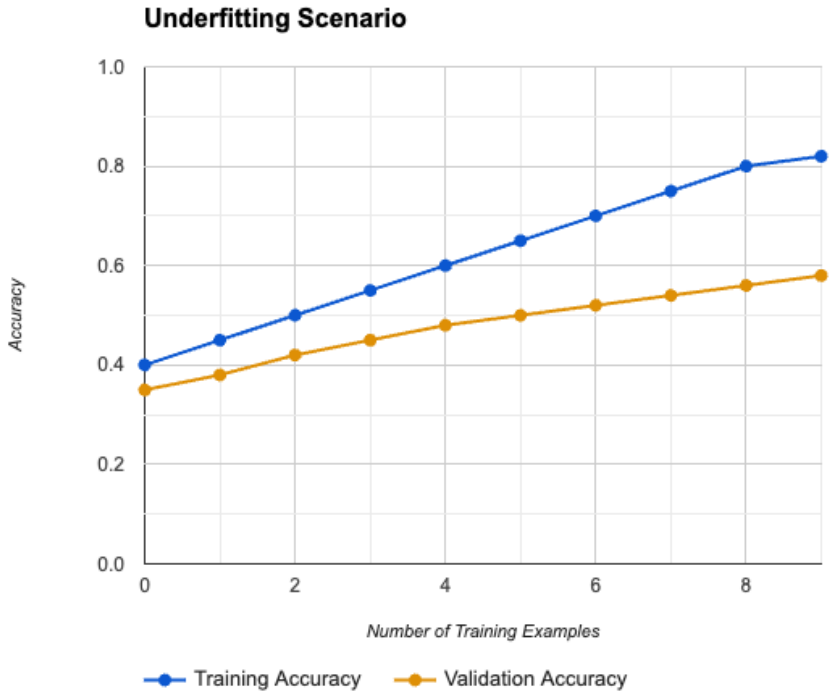
- For large datasets, a simple train-test split might be sufficient, especially if computational resources are limited.
- For smaller datasets, or when the model's performance estimate needs to be as accurate as possible, cross-validation is preferred.
- Choosing between these methods depends on the specific requirements of the project, the size and nature of the dataset, and the computational resources available.

Learning Curves

- Learning curves are graphical representations of a model's performance on the training and validation sets over time or with increasing amounts of training data.
    - **X-axis:** Size of the training dataset or training epochs.
    - **Y-axis:** Performance metric (accuracy, error rate, etc.).
- Analyzing these curves provides valuable insights into:
    - **Model behavior:** Is the model learning effectively? Is it fitting the data well? Is it generalizing well to unseen data?
    - **Hyperparameter tuning:** Which hyperparameter configuration leads to the best performance?
    - **Data requirements:** How much data is needed to achieve a desired level of performance?

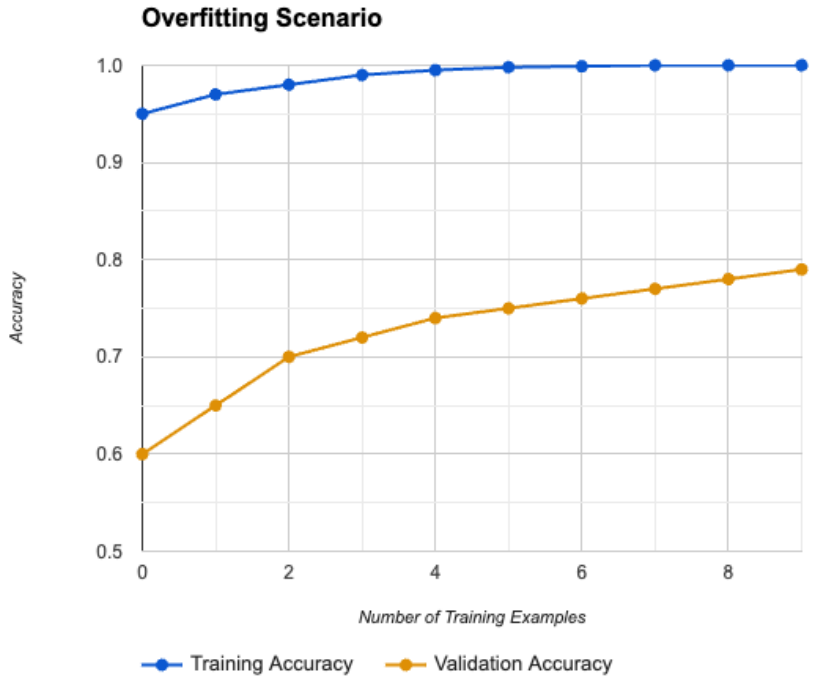
Interpreting Learning Curves

- **Underfitting:**
  - **Characteristics:** Both training and validation scores are low.
  - **Implication:** The model is too simple to capture the data's complexity.
  - **Solution:** Increase model complexity, add features, or decrease regularization.



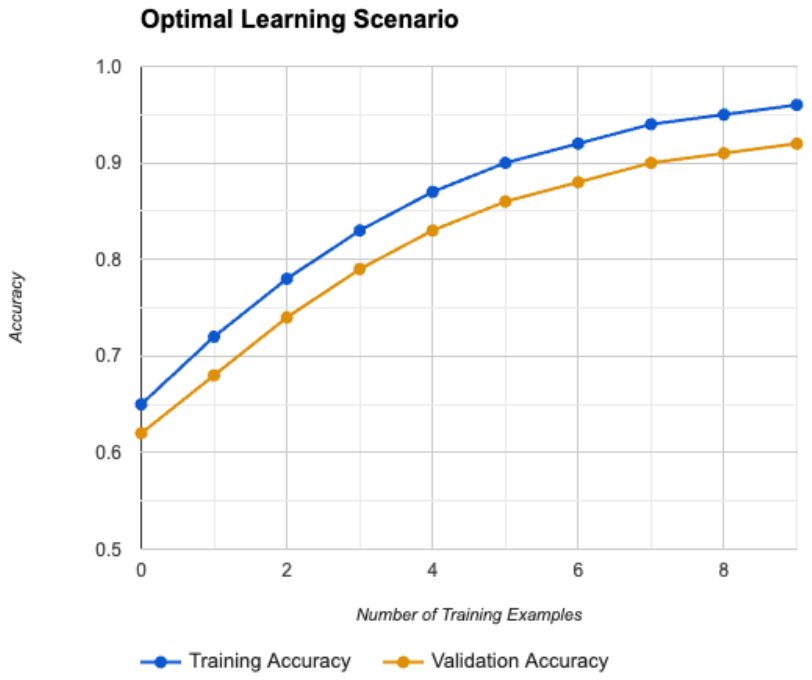
Interpreting Learning Curves

- **Overfitting:**
  - **Characteristics:** High training score but low validation score.
  - **Implication:** The model is too complex, capturing noise in the training data.
  - **Solution:** Simplify the model, add more data, or increase regularization.



Interpreting Learning Curves

- **Ideal (Optimal) Learning:**
  - **Characteristics:** High performance on both training and validation sets, with a narrow gap between them.
  - **Implication:** The model has learned the underlying data pattern well and generalizes to unseen data.
  - **Solution:** Fine-tune and deploy the model.



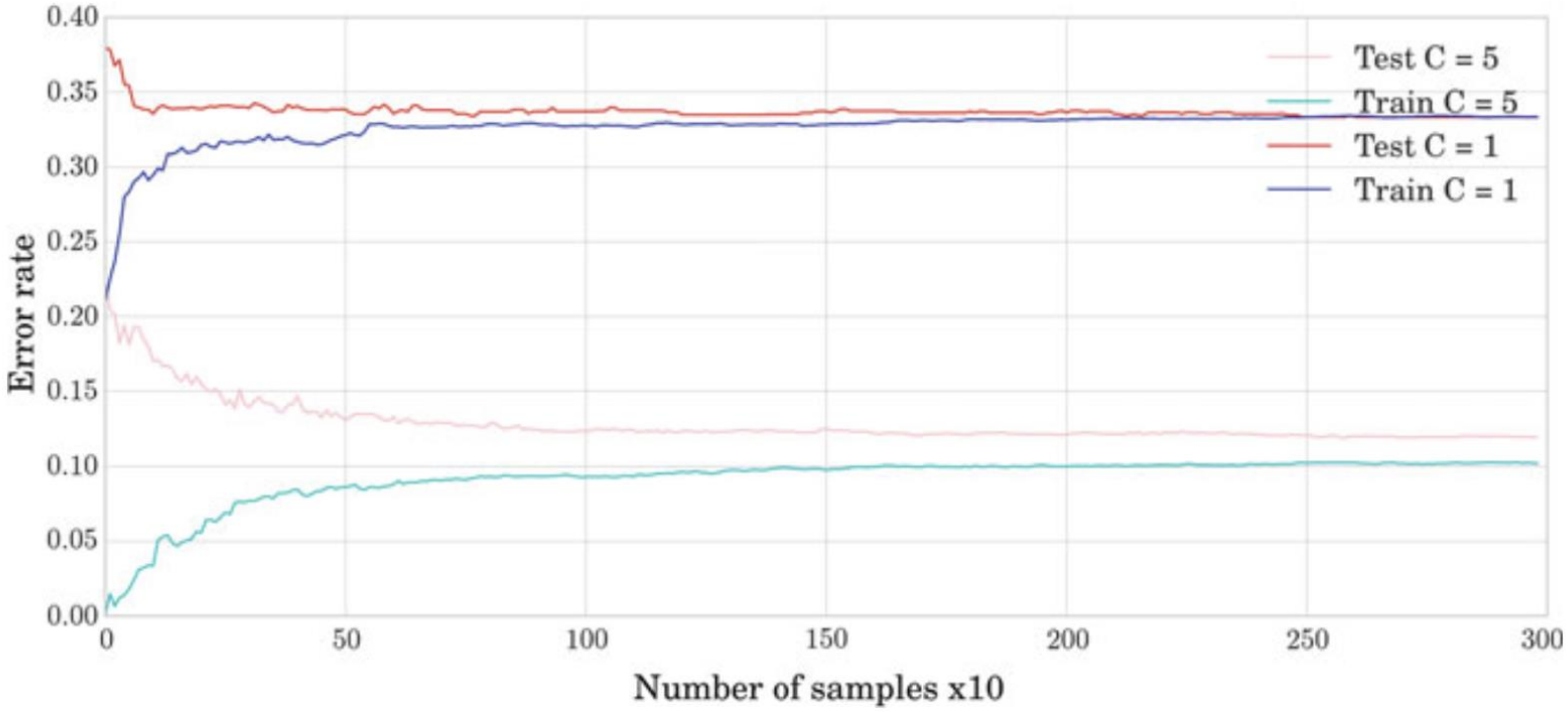


Fig. 5.6 Learning curves (training and test errors) for models with a low and a high degree of complexity

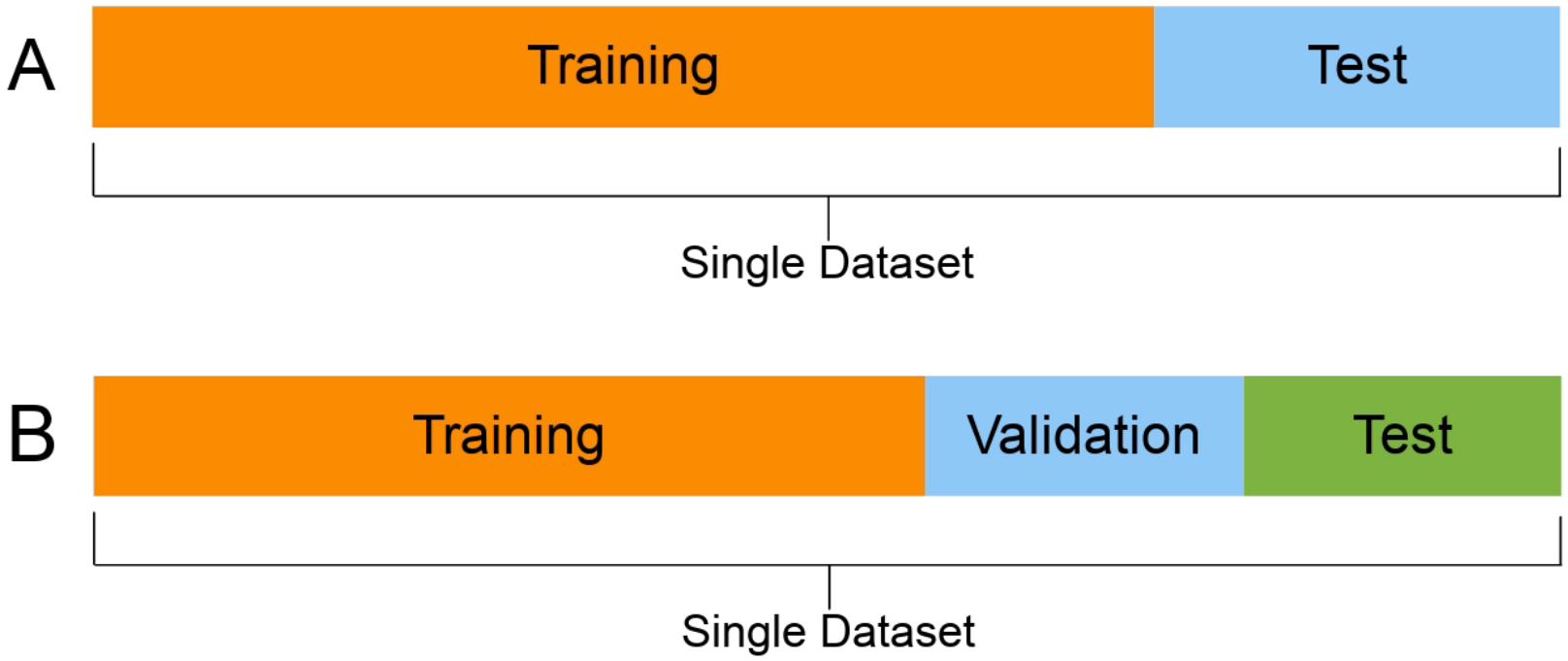
Overfitting and Underfitting: The Two Big problems

- Overfitting:
  - Imagine a student cramming the night before an exam - they remember every detail from the textbook but struggle to apply the knowledge to new problems.
- Underfitting:
  - Imagine a student skipping all lectures and showing up unprepared for the exam - they have no knowledge to apply.
- What about adding '**Validation**' set along with Training and Test?

Training, Validation and Test

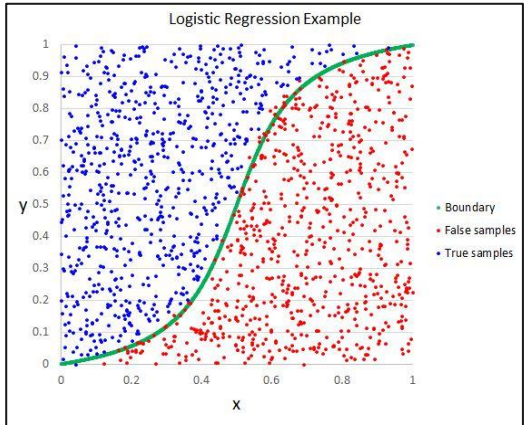
- **Training set:** used to learn the instance of the model from a model class.
- **Validation set:** used explicitly to select the parameters/models with the best performance according to an estimation of the generalization error. This is a form of learning.
- **Test set:** used exclusively for assessing performance at the end of the process and will never be used in the learning process.

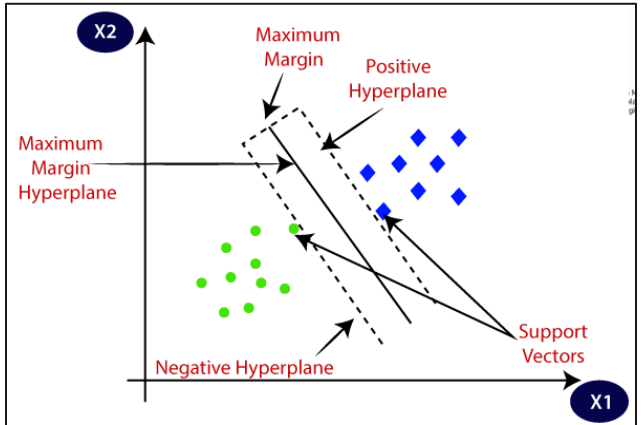
Before and now

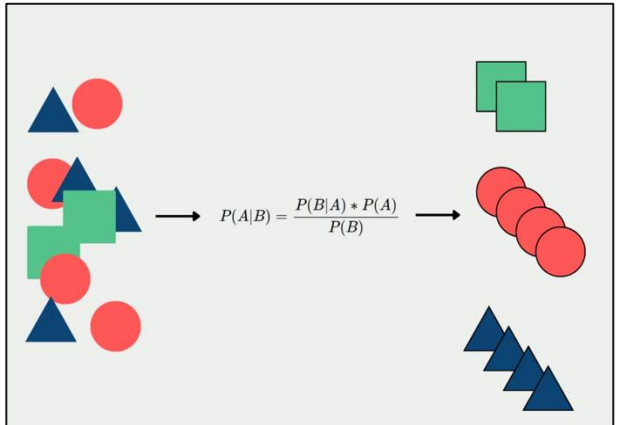


Linear Classifiers

- **Basic Idea:** Find a line or hyperplane that best separates the different classes in your data.
- Popular Examples:
  - **Logistic Regression:** Despite the name, it's a classic classification algorithm predicting probabilities to categories.
  - **Support Vector Machines (SVMs):** Robust classifiers that aim to find the maximum margin hyperplane, increasing the space between classes for better generalization.
  - **Naive Bayes:** Probabilistic classifier using Bayes' Theorem. Simplifies calculations with a strong 'naive' assumption: features are independent of each other.

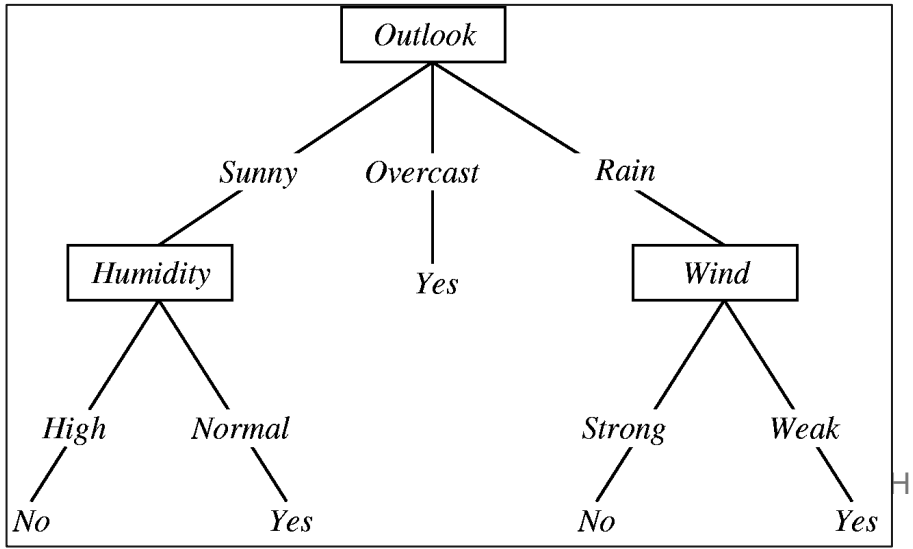


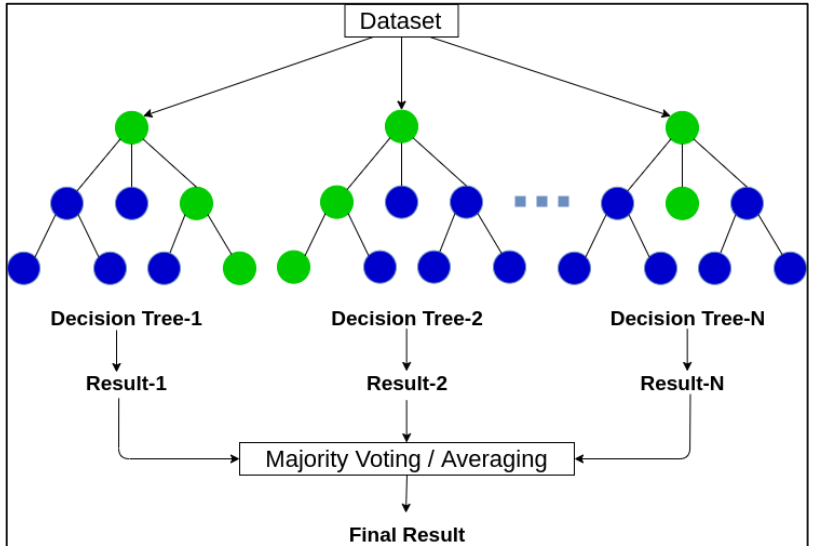




Decision Trees

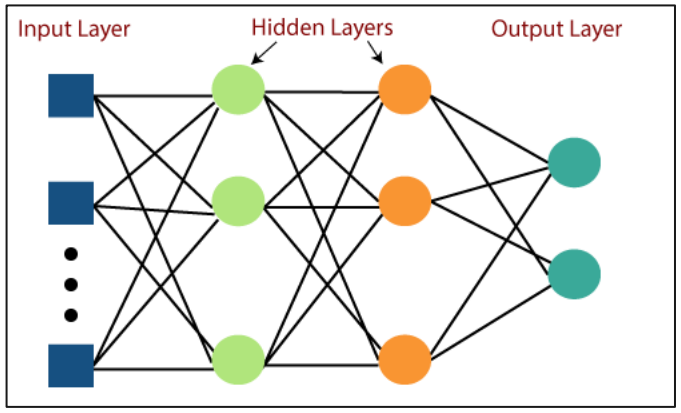
- **Basic Idea:** Build a tree-like flowchart where each node represents a decision based on a feature, and the leaves represent possible class labels.
- Popular Examples:
  - **Decision Trees**: Construct binary trees. Clear visual interpretation of how the model reaches a decision.
  - **Random Forests:** A powerful ensemble method. Builds many decision trees on different data subsets and combines their predictions for improved accuracy. Reduces overfitting issues of single decision trees.

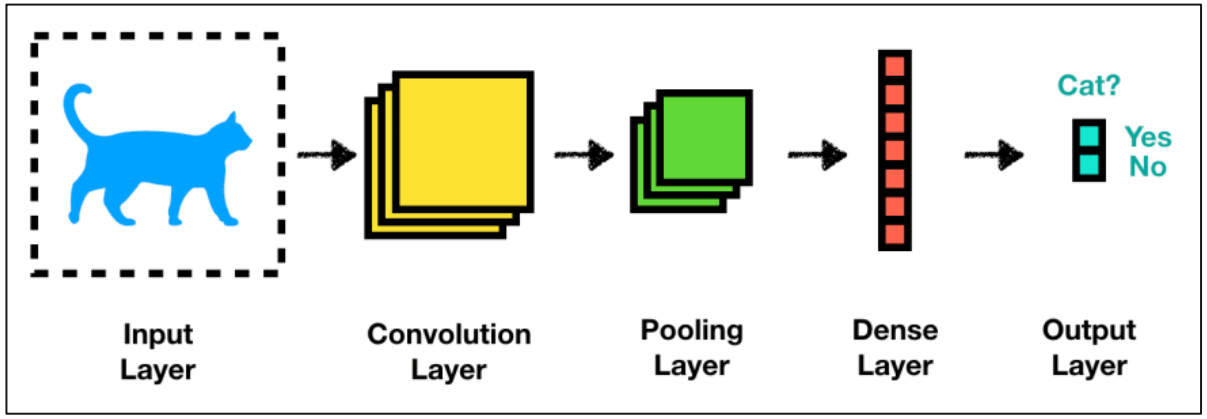




Neural Networks

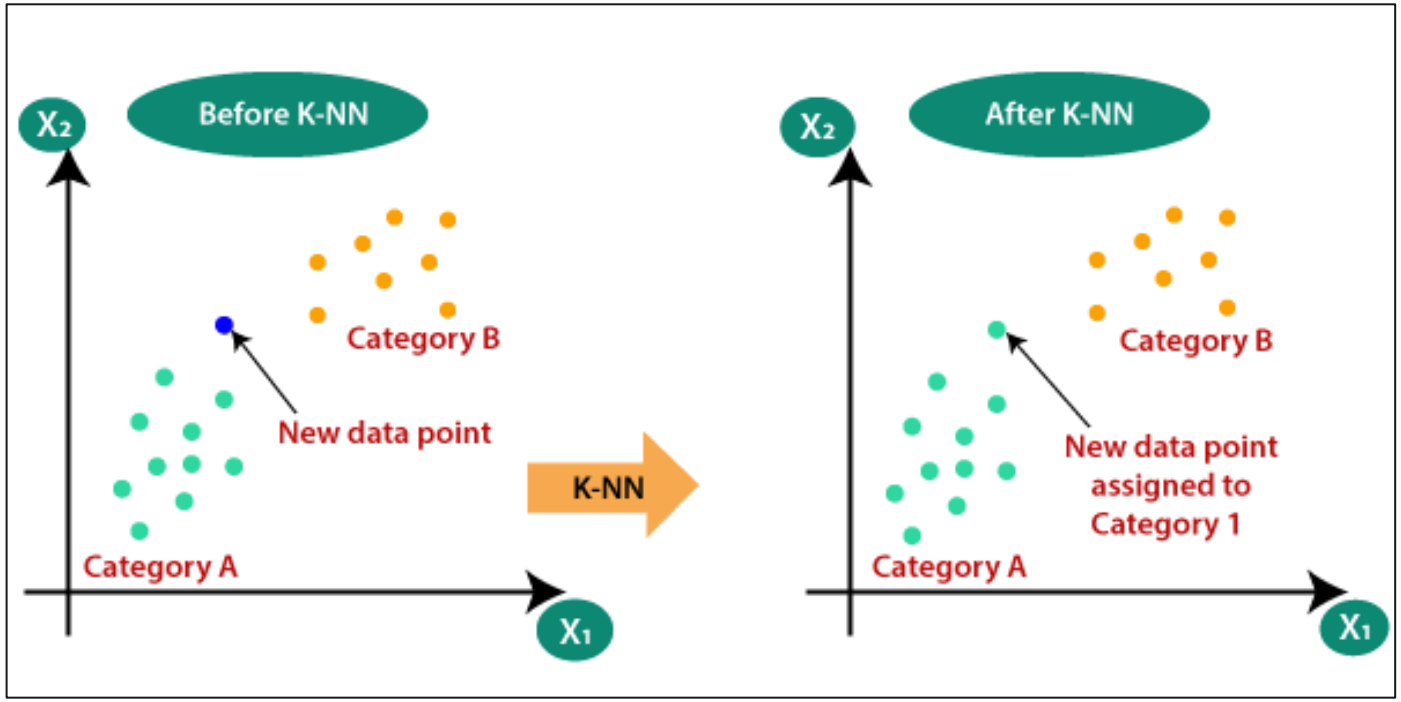
- **Basic Idea:** Inspired by biological brains! A network of interconnected "neurons" process information in layers. Ideal for complex, non-linear relationships in data.
- Popular Examples:
  - **Multilayer Perceptron (MLP):** Basic neural network structure. Popular for classification tasks where there is no defined spatial or sequential structure.
  - **Convolutional Neural Networks (CNNs):** Shine in image classification. Specialized structure to capture spatial patterns in images.





K-Nearest Nighbour (KNN)

- **Basic Idea:** Predicts the class of a new point by finding its 'K' nearest neighbors in the dataset and taking the majority class among those neighbors.
- **Key Traits:** Simple, non-parametric (doesn't make assumptions about underlying data distribution). Effective when you don't have much prior knowledge about the data.



Different categories of Classification algorithms

- Linear vs. Non-Linear:
  - **Linear Classifiers**: Find a straight line (or hyperplane in higher dimensions) to separate classes. [Logistic Regression, Support Vector Machines, Naive Bayes].
  - **Non-Linear Classifiers**: Can create complex, curved decision boundaries to separate classes more effectively for non-linear relationships. [Decision Trees, Random Forests, Neural Networks, Support Vector Machines, KNN]

- Parametric vs. Non-Parametric:
  - **Parametric:** Make assumptions about the underlying data distribution. They learn a fixed set of parameters. [Logistic Regression, Naive Bayes, Linear SVMs]
  - **Non-Parametric:** Do not make strict assumptions about the data's distribution, they adapt their structure based on the data. [Decision Trees, Random Forests, Neural Networks, K-Nearest Neighbors].

- Probabilistic vs Non-Probabilistic:
  - Probabilistic: Predict a probability of a data point belonging to a specific class. [Logistic Regression, Naive Bayes]
  - Non-Probabilistic: Directly predict the class without assigning probabilities. [Decision Trees, SVMs, KNN].